Unscented Kalman Filter for Simultaneous Localization and Mapping: Design and Evaluation

Giuseppe Coppola, 263624

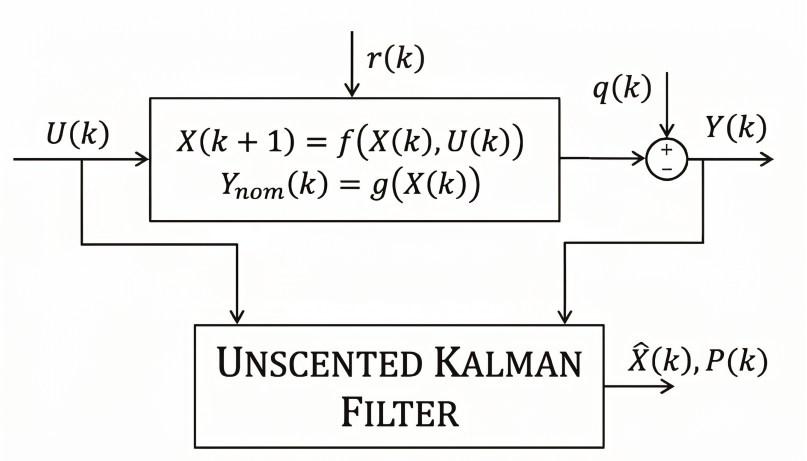
Positioning Systems Course, A.Y. 2025/26

1. Introduction
2. Unscented Kalman Filter (UKF)
3. SLAM
4. UKF-SLAM
5. Application and Simulation

Introduction

- Implement the Unscented Kalman Filter (UKF) as a "derivative-free" alternative to the Extended Kalman Filter (EKF).
- Study the Unscented Transform (UT) and the propagation of uncertainty through sigma points.
- Formulate the SLAM problem as a probablistic estimation problem (pose and map).
- Evaluate the critical role of Data Association (DA).
- Compare the ideal case (perfect IDs) with a realistic case (Mahalanobis Distance).
- Three simulation scenarios: Best Case, Middle Case and Worst Case.

Unscented Kalman Filter (UKF)

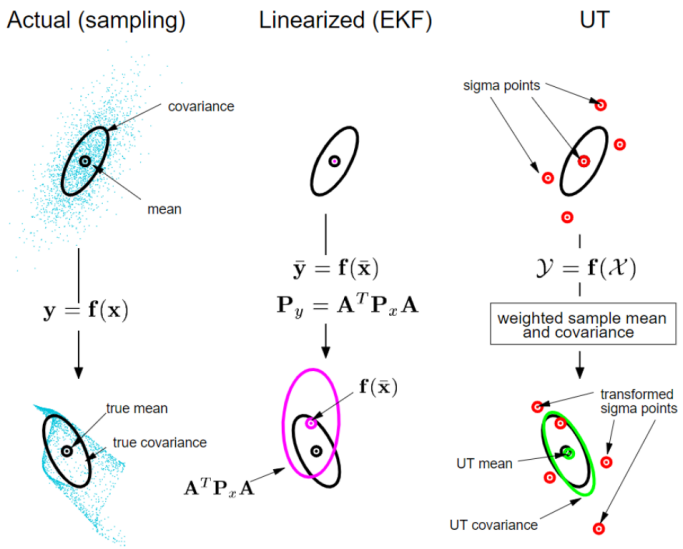


$$\begin{cases} x_{k+1} &= f(x_k, u_k) + r_k \\ y_k &= g(x_k, u_k) + q_k \end{cases} \tag{1}$$

where:

- $x \in \mathbb{R}^n$ is the state, $u \in \mathbb{R}^m$ is the control input, $y \in \mathbb{R}^p$ is the measurement.
- r and q are zero-mean Gaussian white uncorrelated noises with covariance matrices R and Q respectively.

UKF vs EKF



EKF

- Local linearization of the non-linear model.
- Requires the computation of Jacobian matrices.
- Poor accuracy and unrepresentative estimates under strong non-linearities.

UKF

- Derivative-free approach (no Jacobians required).
- Represents probability distribution via deterministic *sigma points*.
- Propagates uncertainty directly through non-linear functions.

Unscented Transform (UT)

For a state of dimension n, we generate $2n+1$ deterministic points:

- $\xi_0 = \bar{x}$
- $\xi_i = \bar{x} + (\sqrt{cP_x})_i$ for $i = 1, \ldots, n$
- $\xi_{i+n} = \bar{x} - (\sqrt{cP_x})_{i-n}$ for $i = n+1, \ldots, 2n$

with a *scaling parameter* $c = \alpha^2(n+\gamma)$:

- α: controls the spread of the points.
- β: incorporates prior knowledge of the distribution shape.

Reconstruction: Points are propagated through the non-linear function $\psi = f(\xi)$, and the new state is estimated using weighted sample mean and covariance.

$$\hat{x} \approx \sum_{i=0}^{2n} w_i^{(m)} \psi_i$$

$$\hat{P}_x \approx \sum_{i=0}^{2n} w_i^{(c)} (\psi_i - \hat{x})(\psi_i - \hat{x})^T$$

UKF Algorithm

1. **Sigma Points and Weights**

   Generation of deterministic points from the a priori state estimate $\hat{x}_{k|k-1}$.

2. **Predicted Measurements and Gain**

   Propagation of sigma points through the measurement model $g(\cdot)$ to compute expected observations, innovation covariance P_y, and Kalman Gain L_k.

3. **Measurement Update**

   Correction of the state $\hat{x}_{k|k}$ and covariance $P_{k|k}$ based on the actual sensor reading y_k.

4. **Time Update (Prediction)**

   Regeneration of sigma points from the updated state and propagation through the dynamic model $f(\cdot)$ to predict the next time step $k+1$.

Algorithm 1 Unscented Kalman Filter (UKF) Non-augmented

Input: $\hat{x}_{0|-1},\ P_{0|-1},\ \alpha,\ \beta,\ \gamma,\ c = \alpha^2(n+\gamma)$

for $k = 0 : N$ **do**
  Sigma points at $k|k-1$
    $\hat{\xi}^{(0)}_{k|k-1} = \hat{x}_{k|k-1}$
  for $i = 1 : n$ **do**
    $\hat{\xi}^{(i)}_{k|k-1} = \hat{x}_{k|k-1} + (\sqrt{cP_{k|k-1}})_i$
    $\hat{\xi}^{(i+n)}_{k|k-1} = \hat{x}_{k|k-1} - (\sqrt{cP_{k|k-1}})_i$
  end for
  Weights
    $w_0^{(m)} = 1 - \frac{n}{c}, \quad w_0^{(c)} = 2 - \alpha^2 + \beta - \frac{n}{c}$
    $w_i^{(m)} = w_i^{(c)} = \frac{1}{2c},\ i = 1, \ldots, 2n$
  Predicted measurements
  for $i = 0 : 2n$ **do**
    $\hat{z}^{(i)}_{k|k-1} = g(\hat{\xi}^{(i)}_{k|k-1}, u_k)$
  end for
    $\hat{y}_{k|k-1} = \sum_{i=0}^{2n} w_i^{(m)} \hat{z}^{(i)}_{k|k-1}$
    $P_y = Q + \sum_{i=0}^{2n} w_i^{(c)} (\hat{z}^{(i)}_{k|k-1} - \hat{y}_{k|k-1})(\hat{z}^{(i)}_{k|k-1} - \hat{y}_{k|k-1})^T$
    $P_{xy} = \frac{1}{2\alpha^2(n+\gamma)} \sum_{i=1}^{2n} (\hat{\xi}^{(i)}_{k|k-1} - \hat{x}_{k|k-1})(\hat{\xi}^{(i)}_{k|k-1} - \hat{x}_{k|k-1})^T$
  Kalman gain
    $L_k = P_{xy} P_y^{-1}$
  Update
    $\hat{x}_{k|k} = \hat{x}_{k|k-1} + L_k(y_k - \hat{y}_{k|k-1})$
    $P_{k|k} = P_{k|k-1} - L_k P_y L_k^T$
  Sigma points at $k|k$
    $\hat{\xi}^{(0)}_{k|k} = \hat{x}_{k|k}$
  for $i = 1 : n$ **do**
    $\hat{\xi}^{(i)}_{k|k} = \hat{x}_{k|k} + (\sqrt{cP_{k|k}})_i$
    $\hat{\xi}^{(i+n)}_{k|k} = \hat{x}_{k|k} - (\sqrt{cP_{k|k}})_i$
  end for
  Prediction
  for $i = 0 : 2n$ **do**
    $\hat{\psi}^{(i)}_{k+1|k} = f(\hat{\xi}^{(i)}_{k|k}, u_k)$
  end for
    $\hat{x}_{k+1|k} = \sum_{i=0}^{2n} w_i^{(m)} \hat{\psi}^{(i)}_{k+1|k}$
    $P_{k+1|k} = R + \sum_{i=0}^{2n} w_i^{(c)} (\hat{\psi}^{(i)}_{k+1|k} - \hat{x}_{k+1|k})(\hat{\psi}^{(i)}_{k+1|k} - \hat{x}_{k+1|k})^T$
end for

Augmented Case

1. **Augmented State Vector ($\hat{x}^{(a)}$)**

   Concatenates the original state vector with process and measurement noise variables.

2. **Augmented Covariance ($P^{(a)}$)**

   A block diagonal matrix combining state covariance ($P_{0|-1}$), process noise covariance (R), and measurement noise covariance (Q).

3. **Sigma Points in Augmented Space**

   Sigma points are generated from the expanded dimension, allowing the non-linear transformation to naturally propagate the uncertainty of both the state and the noise.

Algorithm 2 Unscented Kalman Filter (UKF) Augmented

Input: $\hat{x}^{(a)}_{0|-1} = \begin{bmatrix} \hat{x}_{0|-1} & 0 & 0 \end{bmatrix}^T$, $P^{(a)}_{0|-1} = \begin{bmatrix} P_{0|-1} & 0 & 0 \\ 0 & R & 0 \\ 0 & 0 & Q \end{bmatrix}$, $\alpha,\ \beta,\ \gamma,\ c = \alpha^2(n+\gamma)$

for $k = 0 : N$ **do**

Sigma points at $k|k-1$

$\hat{\xi}^{(0)}_{k|k-1} = \hat{x}^{(a)}_{k|k-1}$

for $i = 1 : n$ **do**

$\hat{\xi}^{(i)}_{k|k-1} = \hat{x}^{(a)}_{k|k-1} + (\sqrt{cP^{(a)}_{k|k-1}})_i$

$\hat{\xi}^{(i+n)}_{k|k-1} = \hat{x}^{(a)}_{k|k-1} - (\sqrt{cP^{(a)}_{k|k-1}})_i$

end for

Weights

$w^{(m)}_0 = 1 - \frac{n}{c}, \quad w^{(c)}_0 = 2 - \alpha^2 + \beta - \frac{n}{c}$

$w^{(m)}_i = w^{(c)}_i = \frac{1}{2c}, \ i = 1, \ldots, 2n$

Predicted measurements

for $i = 0 : 2n$ **do**

$\hat{z}^{(i)}_{k|k-1} = g(\hat{\xi}^{(i)}_{k|k-1}, u_k)$

end for

$\hat{y}^{(a)}_{k|k-1} = \sum_{i=0}^{2n} w^{(m)}_i \hat{z}^{(i)}_{k|k-1}$

$P^{(a)}_y = \sum_{i=0}^{2n} w^{(c)}_i (\hat{z}^{(i)}_{k|k-1} - \hat{y}^{(a)}_{k|k-1})(\hat{z}^{(i)}_{k|k-1} - \hat{y}^{(a)}_{k|k-1})^T$

$P^{(a)}_{xy} = \frac{1}{2\alpha^2(n+\gamma)} \sum_{i=1}^{2n} (\hat{\xi}^{(i)}_{k|k-1} - \hat{x}^{(a)}_{k|k-1})(\hat{\xi}^{(i)}_{k|k-1} - \hat{x}^{(a)}_{k|k-1})^T$

Kalman gain

$L_k = P^{(a)}_{xy} (P^{(a)}_y)^{-1}$

Update

$\hat{x}^{(a)}_{k|k} = \hat{x}^{(a)}_{k|k-1} + L_k (y^{(a)}_k - \hat{y}^{(a)}_{k|k-1})$

$P^{(a)}_{k|k} = P^{(a)}_{k|k-1} - L_k P^{(a)}_y L^T_k$

Sigma points at $k|k$

$\hat{\xi}^{(0)}_{k|k} = \hat{x}^{(a)}_{k|k}$

for $i = 1 : n$ **do**

$\hat{\xi}^{(i)}_{k|k} = \hat{x}^{(a)}_{k|k} + (\sqrt{cP^{(a)}_{k|k}})_i$

$\hat{\xi}^{(i+n)}_{k|k} = \hat{x}^{(a)}_{k|k} - (\sqrt{cP^{(a)}_{k|k}})_i$

end for

Prediction

for $i = 0 : 2n$ **do**

$\hat{\psi}^{(i)}_{k+1|k} = f(\hat{\xi}^{(i)}_{k|k}, u_k)$

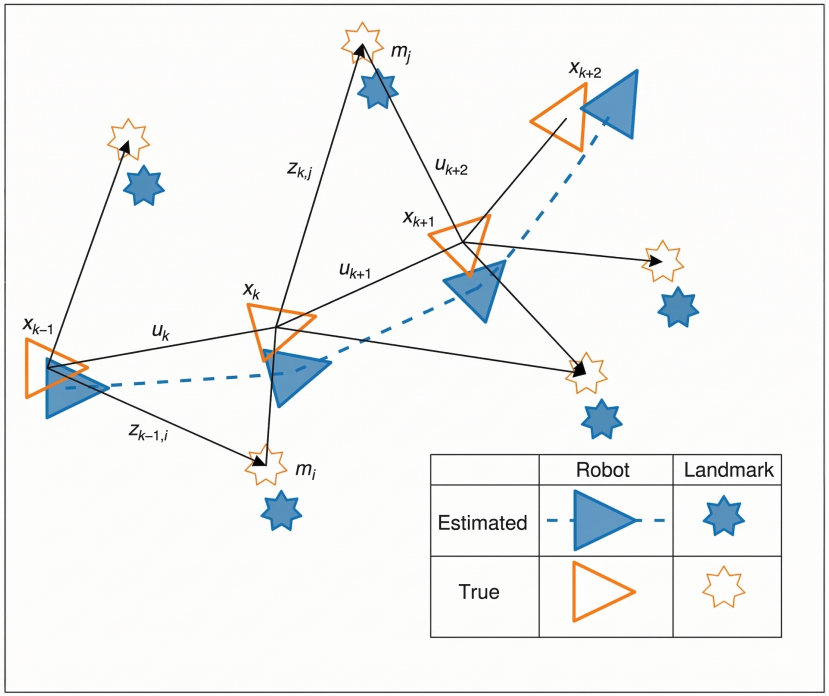
end for

$\hat{x}^{(a)}_{k+1|k} = \sum_{i=0}^{2n} w^{(m)}_i \hat{\psi}^{(i)}_{k+1|k}$

$P^{(a)}_{k+1|k} = \sum_{i=0}^{2n} w^{(c)}_i (\hat{\psi}^{(i)}_{k+1|k} - \hat{x}^{(a)}_{k+1|k})(\hat{\psi}^{(i)}_{k+1|k} - \hat{x}^{(a)}_{k+1|k})^T$

end for

SLAM



- **The Problem**: Enabling a mobile robot to build a map of an unknown environment while determining its own position within it.
- No a priori knowledge of the location is assumed.
- **Probabilistic Formulation**: SLAM is a joint estimation problem. The goal is to compute the joint posterior density:

$$p(x_k, m | z_{0:k}, u_{0:k}, x_0)$$

Recursive Solution

The process consists into two main steps using **Bayes' Theorem**:

1. **Prediction (Time Update)**

$$p(x_k, m|z_{0:k-1}, u_{0:k-1}) = \int p(x_k|x_{k-1}, u_{k-1}) p(x_{k-1}, m|z_{0:k-1}, u_{0:k-1}, x_0) dx_{k-1}$$

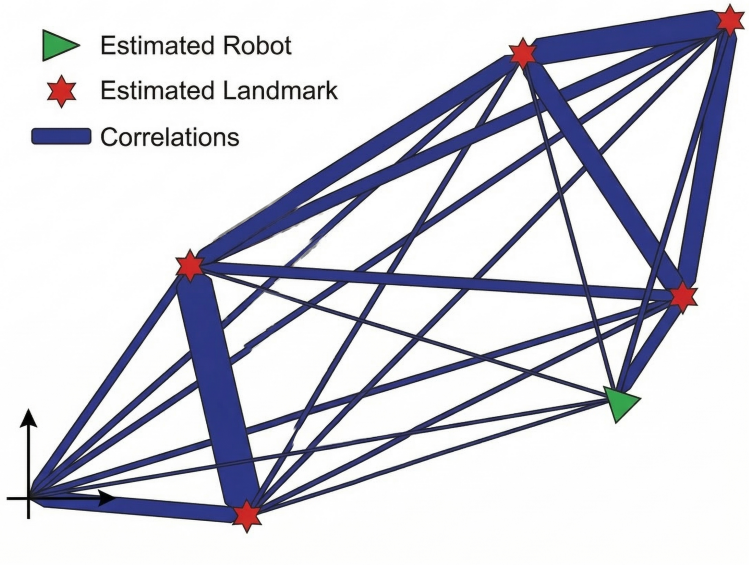
- Uses the state transition (motion) model.
- Based on the **Markov assumption**: the state transition depends only on the immediately preceding state x_{k-1} and the applied control u_{k-1}.

2. **Estimation (Measurement Update)**

$$p(x_k, m|z_{0:k}, u_{0:k-1}, x_0) = \frac{p(z_k|x_k, m) p(x_k, m|z_{0:k-1}, u_{0:k-1}, x_0)}{p(z_k|z_{0:k-1}, u_{0:k-1})}$$

- Uses the observation (sensor) model.
- Assumes observations z_k are conditionally independent given the map and the current robot state.

Convergence: Correlations and The Spring Analogy



- **Common Error Source**: Estimated landmark positions are strongly correlated because they share a single error source: the robot's pose uncertainty during observation.
- **Probabilistic Insight**: While the absolute marginal density $p(m_i)$ may be highly dispersed, the joint probability density $p(m_i, m_j)$ (relative location) is highly peaked.
- **Monotonic Convergence**: Correlations between landmark estimates increase monotonically as observations act as nearly independent measurements of relative distances.

Convergence: Correlations and The Spring Analogy

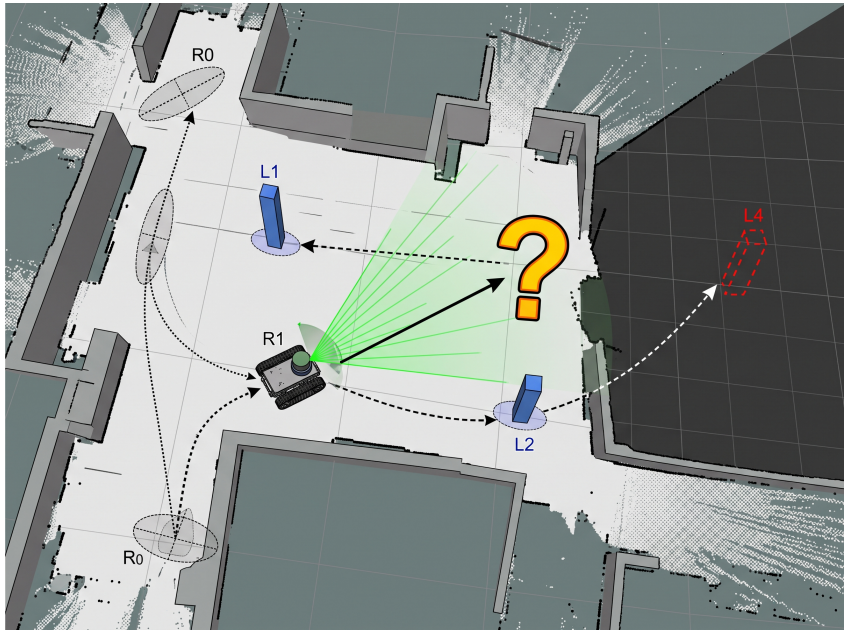
The **Convergence Mechanism** consists in:

- **Chain Propagation**
  Updating an observed landmark mathematically propagates the update to unseen correlated landmarks.

- **Network Creation**
  Every newly observed landmark is linked to the existing interconnected network.

- **Spring Analogy**
  Each robot observation is a "pull" that progressively stiffens local springs and propagates the effect throughout the map.

Data Association Problem



- **The Core Issue**: Assigning anonymous sensor measurements to the correct map features.
- **The Decision**: When the robot observes a feature, it must determine if it is a *previously mapped landmark* or a *newly discovered one*.
- **The Catastrophic Risk**: Classical SLAM systems are fragile. A single incorrect association irreversibly corrupts the augmented covariance matrix, leading to divergence of both map and pose estimation.

Data Association Problem

The main strategies for Robust Data Association are:

1. **Batch Validation (Geometric Consistency)**
   - Overcomes the unreliability of individual gating by evaluating sets of associations simultaneously to ensure mutual geometric compatibility.
   - *JCBB* (Joint Compatibility Branch and Bound): Performs a tree search to find the largest set of jointly compatible associations.
   - *CCDA* (Combined Constraint Data Association): Constructs a correspondence graph and solves the problem by finding a clique of geometrically compatible pairs.
2. **Appearance Signatures**
   - Leverages rich sensors (e.g., cameras) to extract visual features such as shape, color, and texture.
   - Crucial for detecting loop closures and providing highly discriminative information that enhances the robustness of geometric methods.
3. **Multiple Hypothesis Tracking (MHT)**
   - Originating from target tracking in cluttered environments, MHT delays ambiguous decisions rather than committing immediately.
   - Generates multiple association hypotheses, forming a decision tree where each branch maintains a separate map and pose estimate.
   - Low-probability hypotheses are gradually pruned to manage computational load.

UKF-SLAM

Given the model (1), it can be defined the **dynamic augmented state vector**:

$$\zeta_k = \begin{bmatrix} x_k \\ m \end{bmatrix}$$

- **STEP 1:Prediction Step**
  The robot pose x_k is propagated through the motion model. The map m remains static, but process noise increases overall uncertainty.
- **STEP 2: Observation and Data Association**
  The sensor detects a feature z_k. The algorithm determines if it corresponds to an existing landmark in m or a completely new entity.

UKF-SLAM

- **STEP 3a: Measurement Update (Known Landmark)**
  If the observation matches the map, the filter corrects the joint state, reducing uncertainty for both the robot and the map.
- **STEP 3b: Map Augmentation (New Landmark)**
  If the feature is unassociated, its global coordinates m_{new} are computed using the robot's pose and the inverse observation model.

The state vector ζ and the augmented covariance matrix $\mathcal{P}$ expand dynamically to include the new feature:

$$\zeta_{new} = \begin{bmatrix} \zeta \\ m_{new} \end{bmatrix}$$

$$\mathcal{P}_{new} = \begin{bmatrix} \mathcal{P} & \mathcal{P}_{\zeta, m_{new}} \\ \mathcal{P}_{m_{new}, \zeta} & \mathcal{P}_{m_{new}, m_{new}} \end{bmatrix}$$

UKF-SLAM

Data Association: Mahalanobis and Gating Logic

The **Mahalanobis Metric** normalizes the measurement error (ν) dynamically using the innovation covariance matrix (S).

$$D^2 = \nu^T S^{-1} \nu$$

The **Validation Gate** (γ): Threshold set at $\gamma = 5.991$, representing the 95% confidence region of a χ^2 distribution (2 degrees of freedom).

How the Algorithm Works (Step-by-Step):

1. For each new sensor reading z_k, compute D^2 against all previously mapped landmarks.
2. Identify the candidate with the minimum distance: $\min(D^2)$.
3. Evaluate the Gate:
   - If $\min(D^2) < \gamma \rightarrow$ **Valid Match**: The reading belongs to the known landmark (triggers UKF Update).
   - If $\min(D^2) \geq \gamma \rightarrow$ **Rejection**: The reading is statistically incompatible with the map (triggers New Landmark Initialization).

Simulation Environment and Robot Model

- **Environment**: 2D plane with randomly distributed static landmarks.
- **Kinematic Model (Differential Drive)**: The robot state is $X = [x, y, \theta]^T$, driven by odometry control inputs $U = [t, r_1, r_2]^T$.

$$\begin{cases} x_{k+1} &= x_k + t \cdot \cos(\theta_k + r_1) \\ y_{k+1} &= y_k + t \cdot \sin(\theta_k + r_1) \\ \theta_{k+1} &= \theta_k + r_1 + r_2 \end{cases} \iff x_{k+1} = f(x_k, u_k)$$

- **Observation Model (Range-Bearing Sensor)**: Computes expected distance (r) and relative angle (ϕ) to a landmark at (m_x, m_y).

$$\begin{cases} r &= \sqrt{(m_x - x)^2 + (m_y - y)^2} \\ \phi &= \arctan 2(m_y - y, m_x - x) - \theta \end{cases} \iff y_k = g(x_k)$$

Simulation Environment and Robot Model

- **Non-linear System Formulation**

$$\begin{cases} x_{k+1} &= f(x_k, u_k) + r_k \\ y_k &= g(x_k) + q_k \end{cases}$$

- **Process Noise (r_k)**: Zero-mean Gaussian noise with covariance R (depends on a variable scalar (σ_{noise}) to test different difficulty levels).

$$R = \begin{bmatrix} \sigma_{noise} & 0 & 0 \\ 0 & \sigma_{noise} & 0 \\ 0 & 0 & \frac{\sigma_{noise}}{10} \end{bmatrix}$$

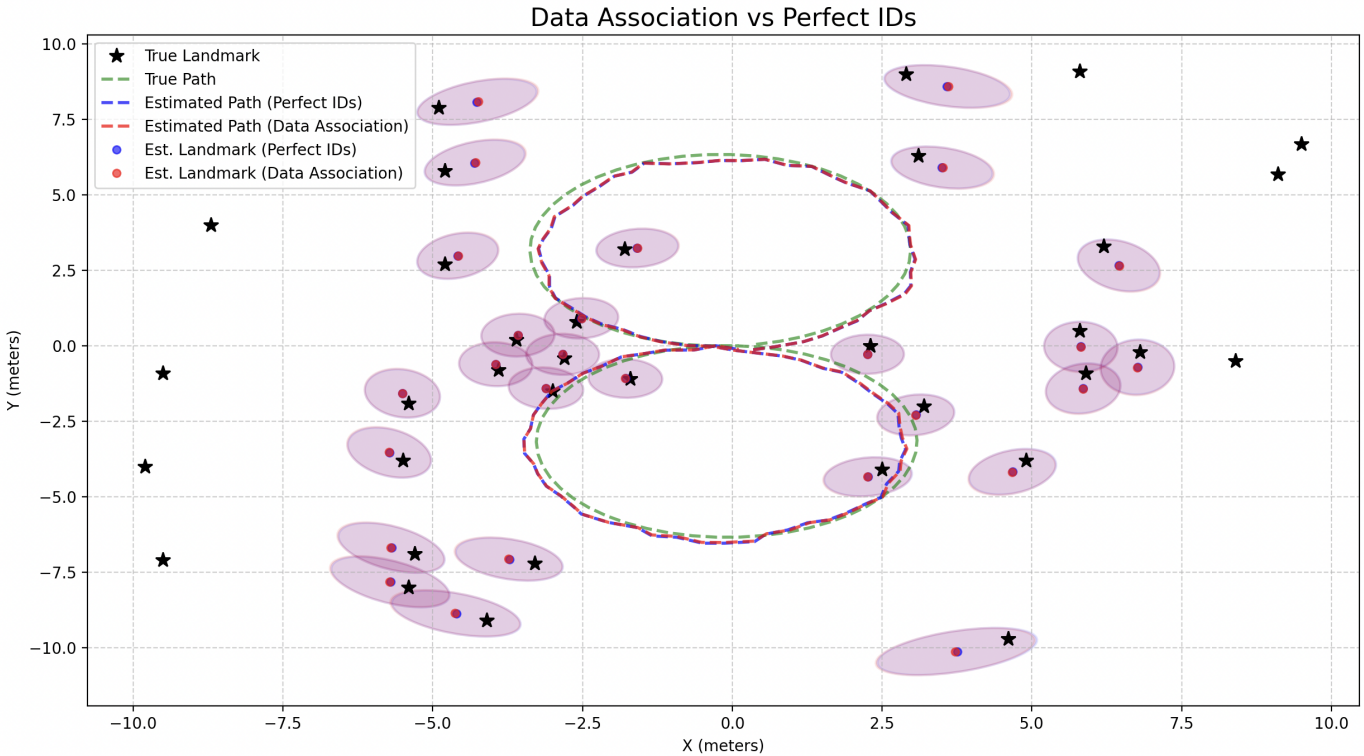
- **Measurement Noise (q_k)**: Zero-mean Gaussian noise with covariance Q.

$$Q = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.01 \end{bmatrix}$$

Simulation Scenarios and Evaluation Methodology

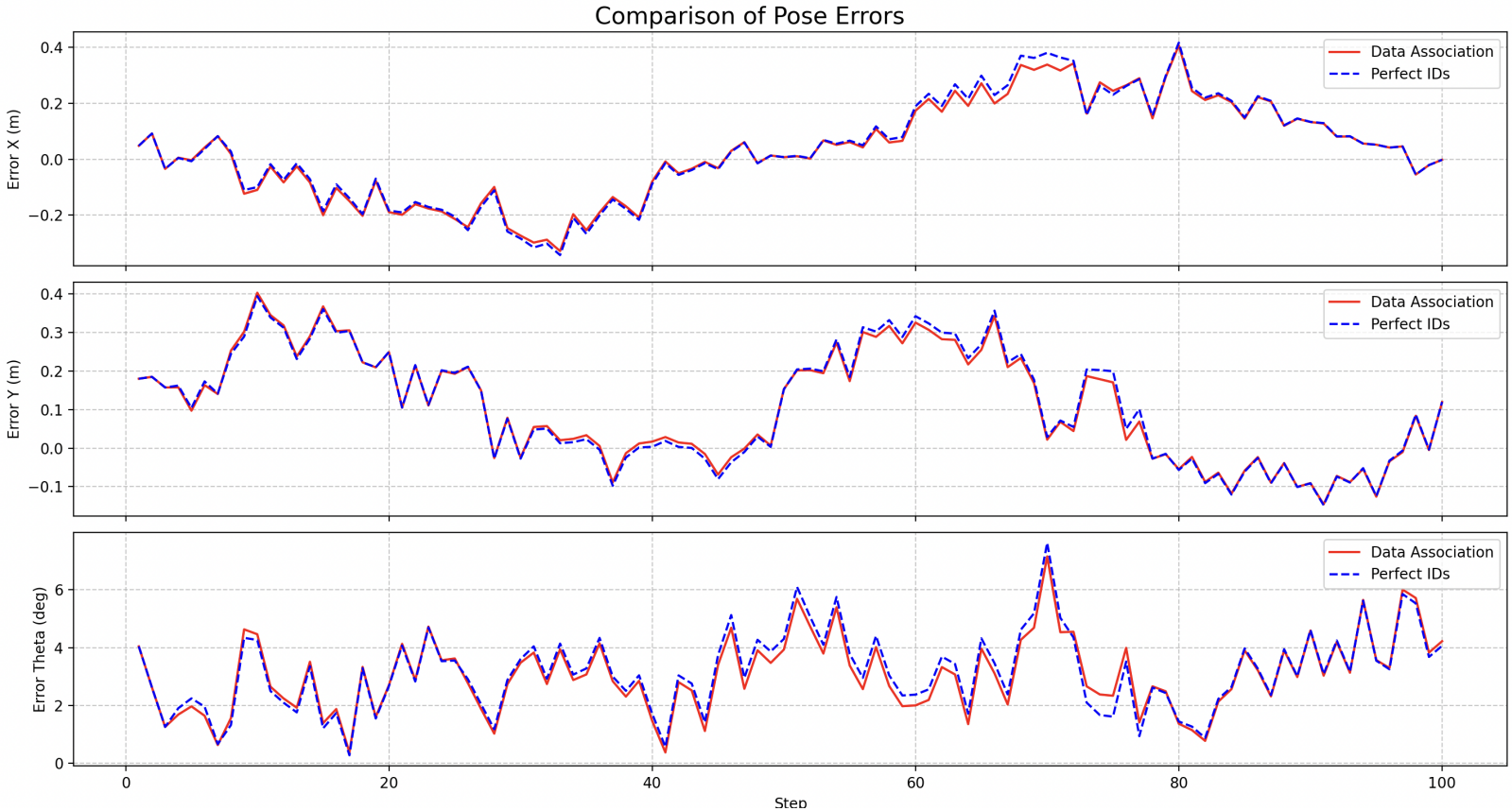
- **Evaluation Approach**: Simultaneous execution comparing the ideal baseline (*Perfect IDs*) against the realistic implementation (*Mahalanobis Data Association*).
- **Scenario 1: BEST CASE**
  - *Environment*: High feature density (35 Landmarks).
  - *Kinematic Uncertainty*: Very low process noise ($\sigma_{noise} = 0.05$).
- **Scenario 2: MIDDLE CASE**
  - *Environment*: Standard distribution (16 Landmarks).
  - *Kinematic Uncertainty*: Moderate process noise ($\sigma_{noise} = 0.15$).
- **Scenario 3: WORST CASE**
  - *Environment*: Highly sparse (4 Landmarks).
  - *Kinematic Uncertainty*: Severe process noise ($\sigma_{noise} = 0.20$). Designed to stress-test the gating logic and force potential divergence.

Scenario 1: BEST CASE



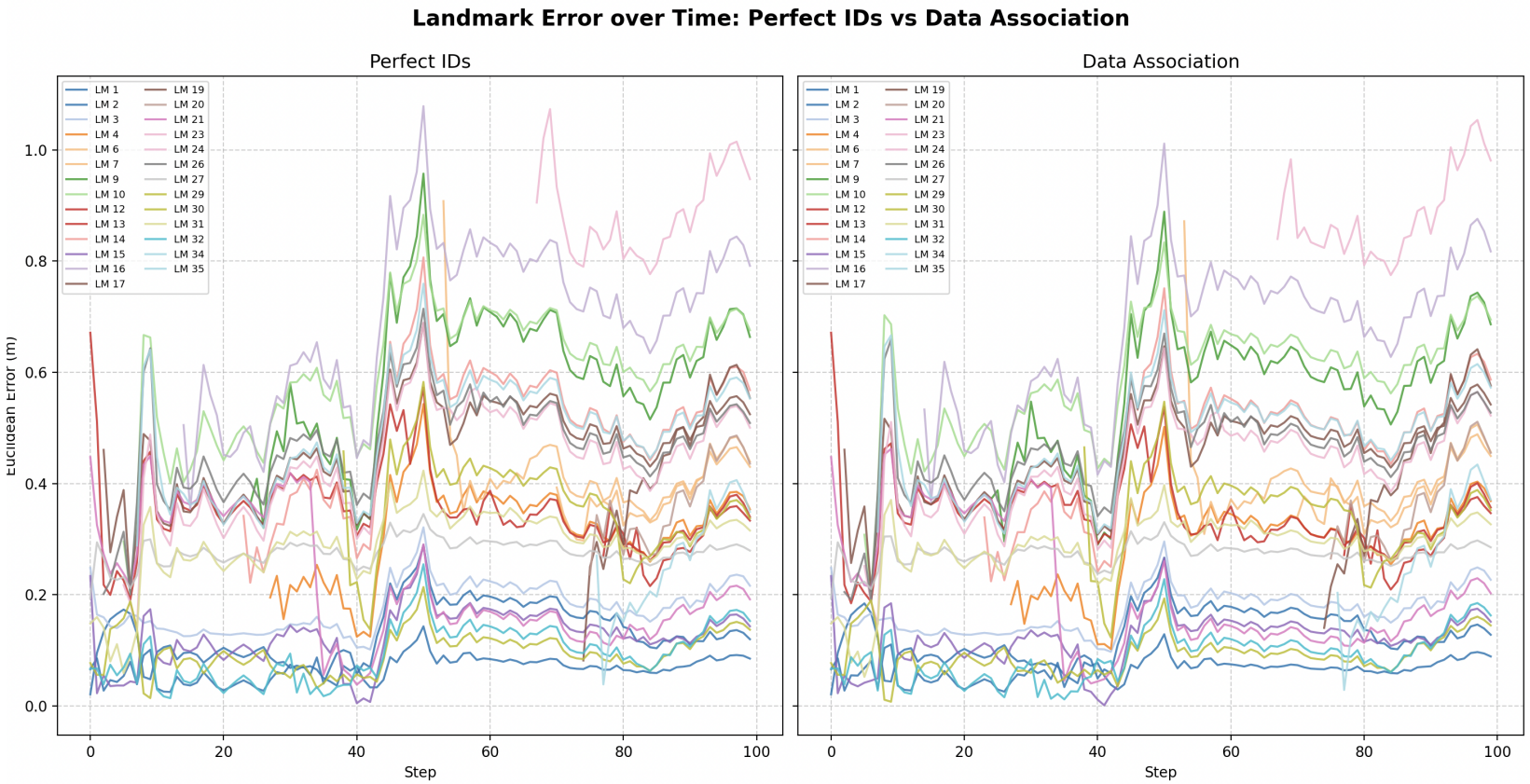
- **Trajectory Tracking**: The Data Association estimated path (red dashed line) perfectly overlaps the ideal Perfect IDs path (blue dashed line) and tracks the true path (green).
- **Map Consistency**: All 35 landmarks are correctly associated (red dots) without generating false positives, enclosed by stable confidence ellipses (pink).

Scenario 1: BEST CASE



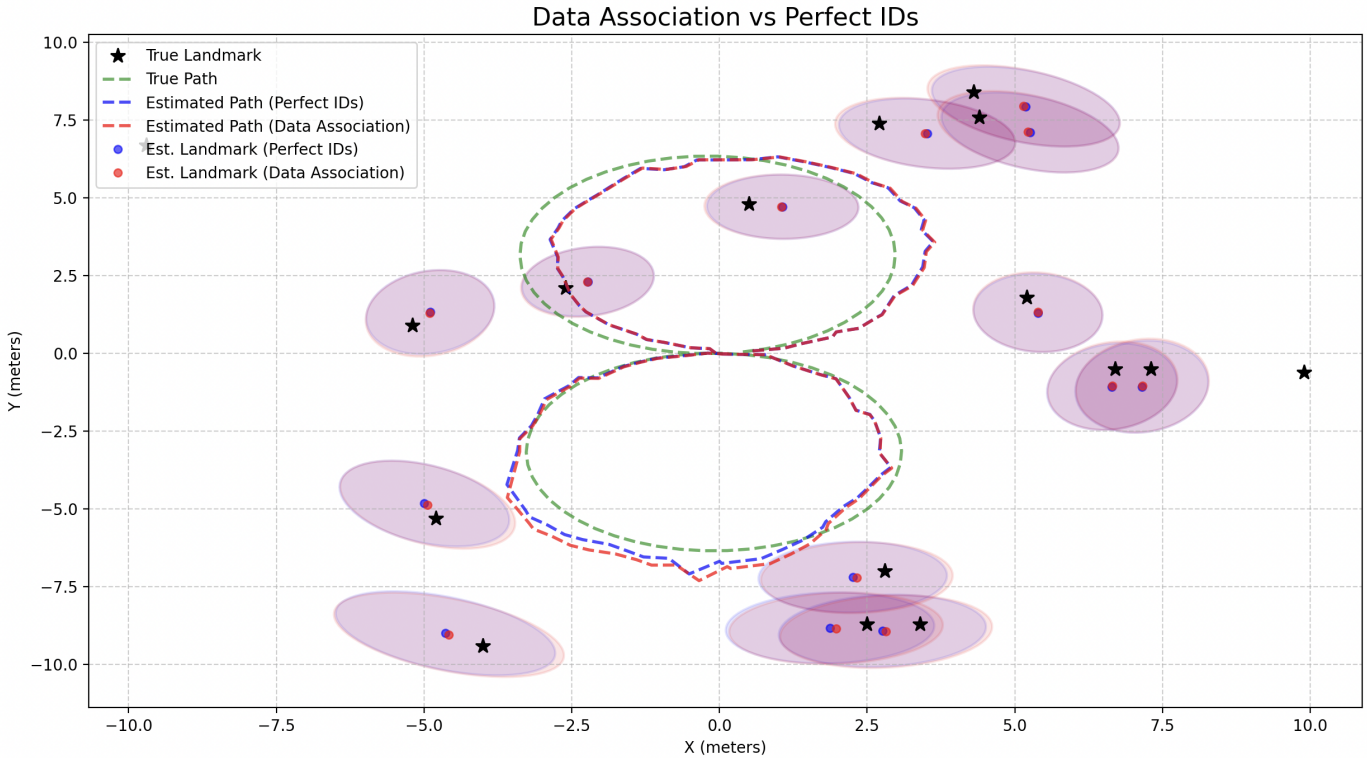
- **Performance Parity**: The Data Association error (red solid line) tightly bounds to the Perfect IDs error (blue dashed line) in all three spatial dimensions.
- **Stability**: Despite odometry noise integration, maximum positional errors remain heavily bounded (e.g., < 0.4 meters in X and Y).

Scenario 1: BEST CASE



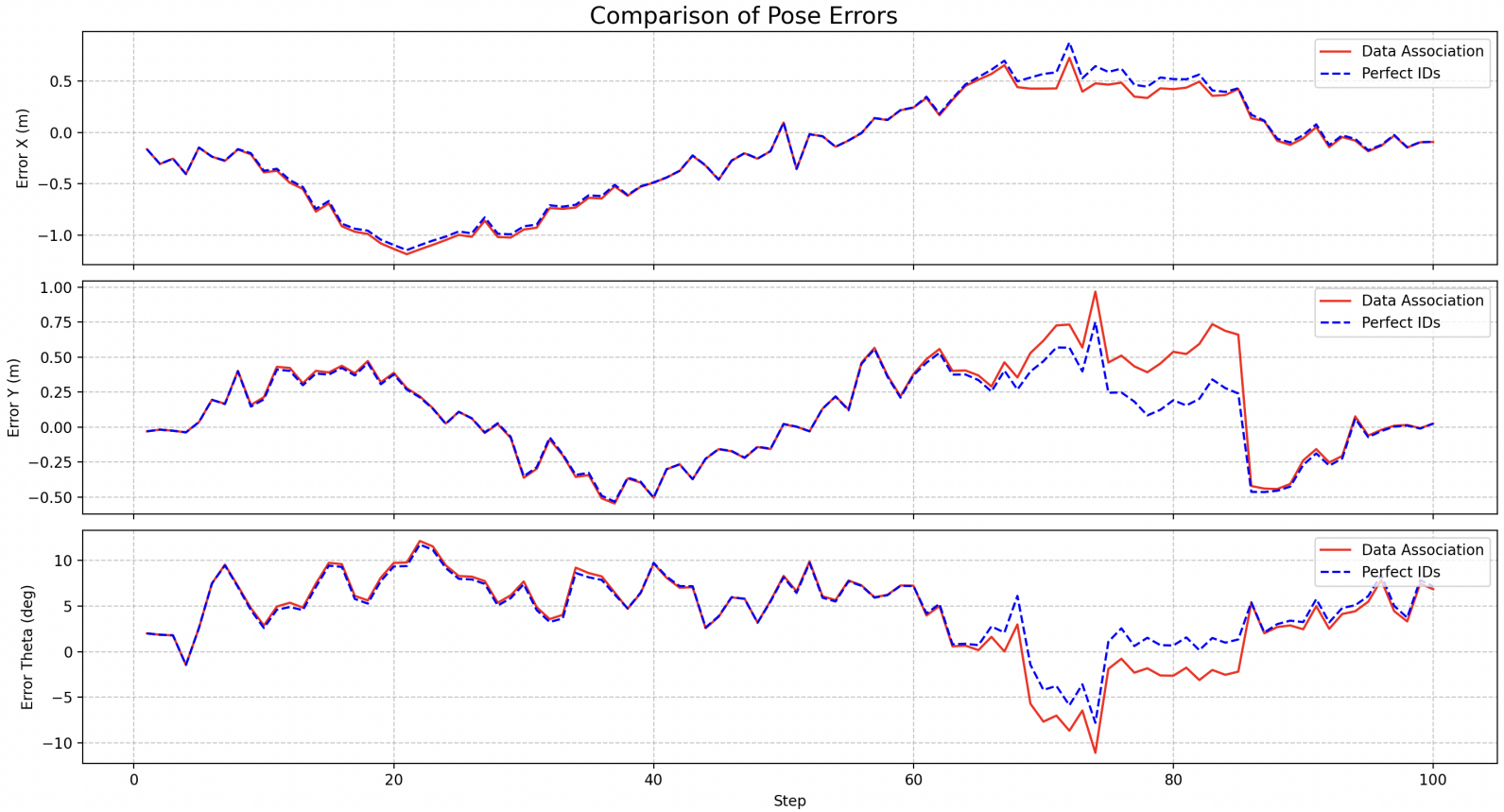
- **Convergence Patterns**: Identical error reduction behavior is observed between the Perfect IDs baseline (left) and the Data Association implementation (right).
- **Correlation Effect**: Sharp drops in error correlate with successful feature observations, demonstrating effective map rigidification.

Scenario 2: MIDDLE CASE



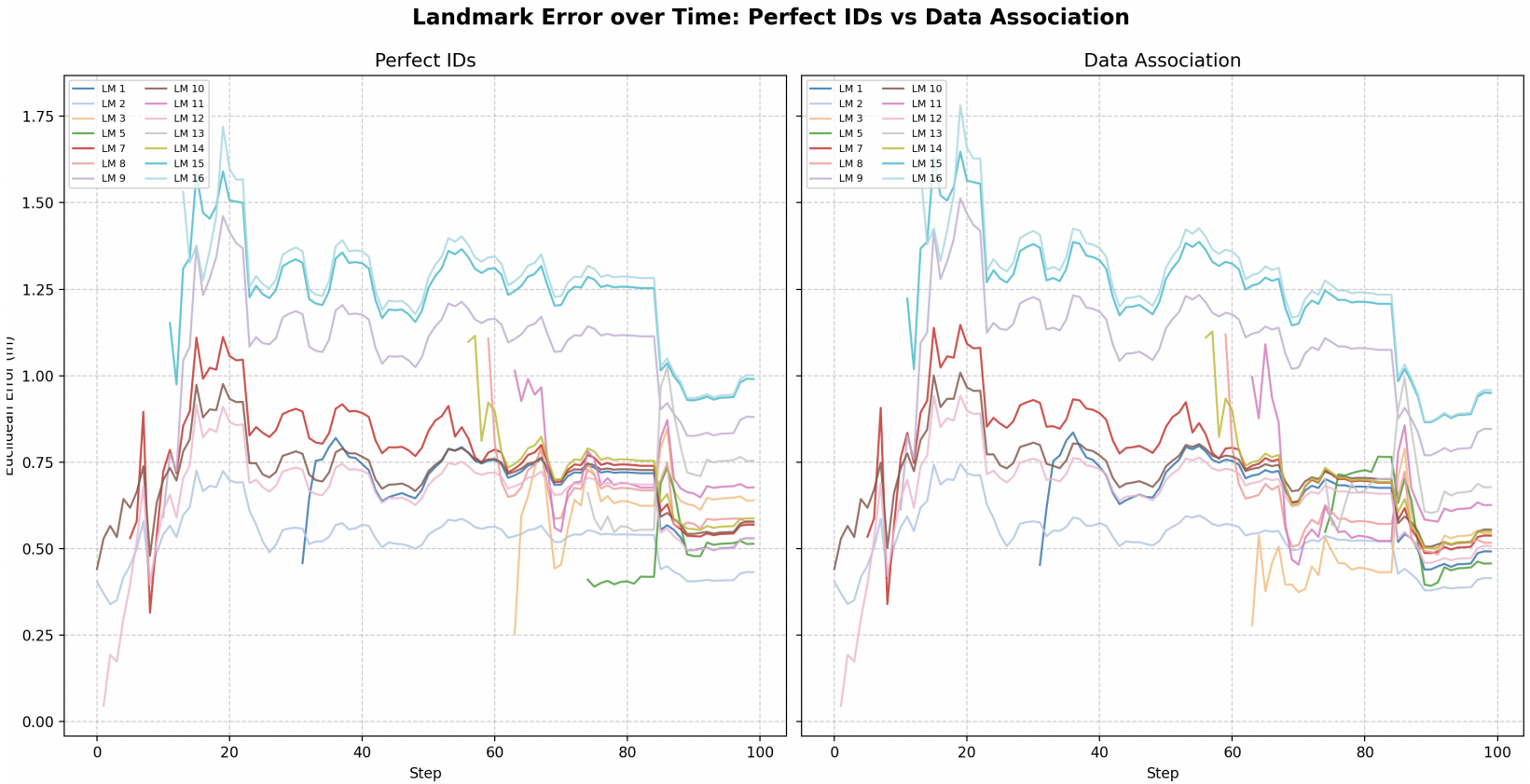
- **Trajectory Tracking**: The Data Association estimated path (red dashed line) begins to show minor, brief deviations from the ideal Perfect IDs path (blue dashed line) due to increased noise.
- **Map Consistency**: Despite the increased uncertainty, the system correctly associates all 16 landmarks without any false positives. The confidence ellipses correctly encompass the true landmark positions.

Scenario 2: MIDDLE CASE



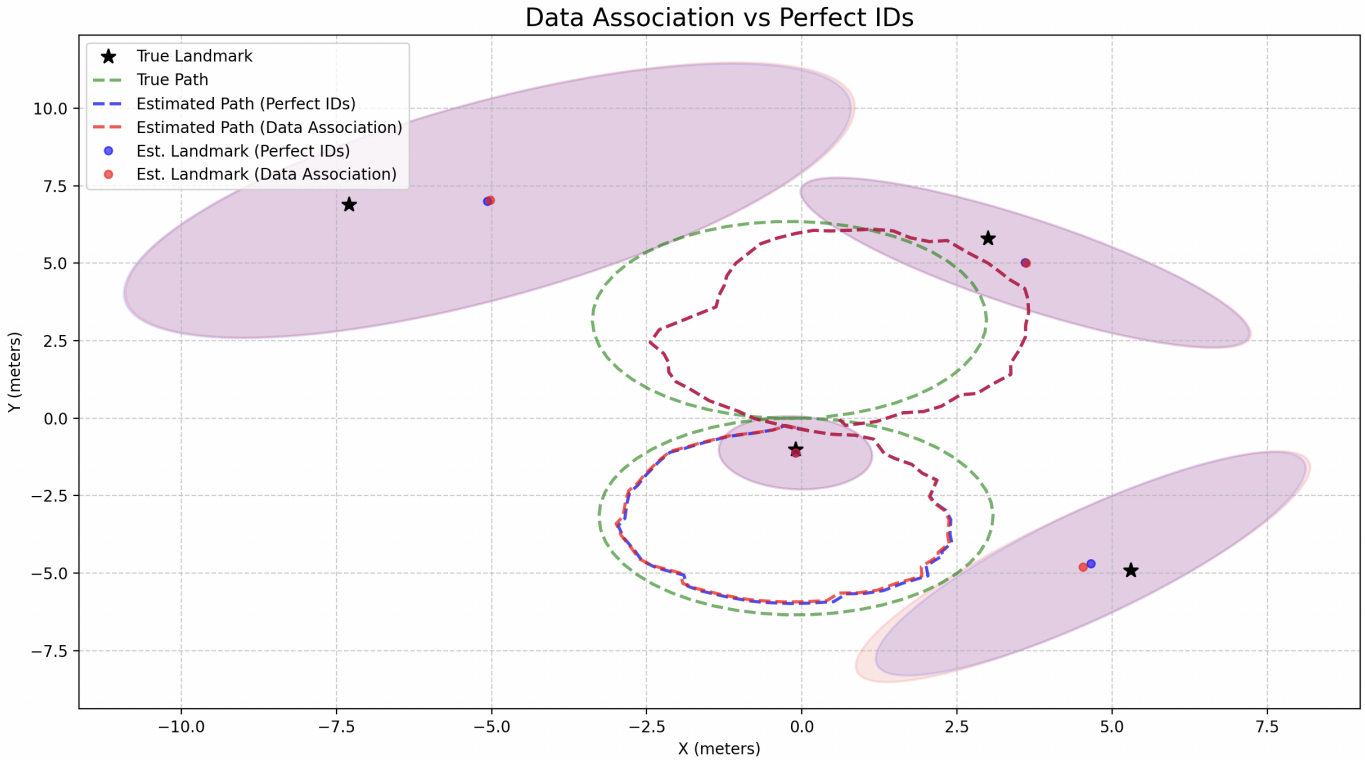
- **Deviation Emergence**: Around step 70, the Data Association error (red solid line) temporarily diverges from the Perfect IDs baseline (blue dashed line) along the Y-axis and Theta.
- **Recovery**: The gating logic remains robust; as new landmarks enter the field of view (around step 85), the filter successfully corrects the pose and snaps back to baseline performance.

Scenario 2: MIDDLE CASE



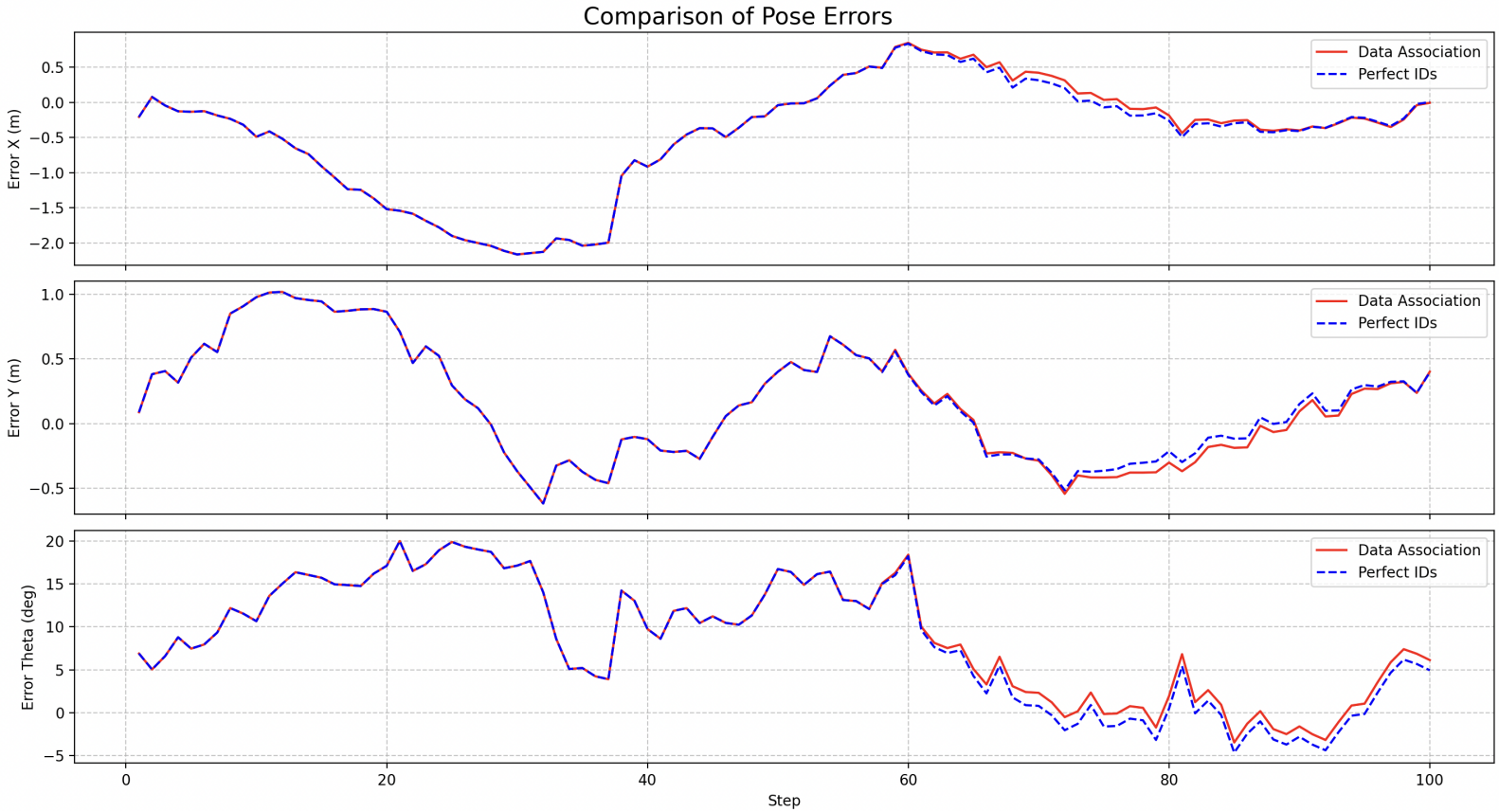
- **Convergence Maintenance**: Both algorithms exhibit the same monotonic error reduction over time.
- **Robust Data Association**: The Data Association plot (right) perfectly mirrors the Perfect IDs plot (left), proving that the Mahalanobis gating logic did not misclassify any features despite the severe pose perturbations.

Scenario 3: WORST CASE



- **Uncertainty Visualization**: The massive confidence ellipses (pink) visually demonstrate the extreme degradation of the system's spatial certainty.
- **Survival Over Divergence**: Despite the extreme drift, the Data Association path (red dashed line) successfully completes the loop without catastrophic failure. All 4 landmarks are identified without generating false positives.

Scenario 3: WORST CASE



- **Performance Gap**: In the final 30 steps, a visible gap emerges between the Data Association estimation (red solid line) and the Perfect IDs baseline (blue dashed line).
- **System Resilience**: Although the Data Association filter struggles slightly more than the ideal baseline, it does not diverge completely and maintains tracking.

Scenario 3: WORST CASE



- **Absolute Robustness**: The Data Association convergence plot (right) remains completely identical to the Perfect IDs baseline (left).
- **Final Conclusion**: The Mahalanobis gating logic never misclassified a feature, proving highly robust even when system covariance (S) was enormously inflated.